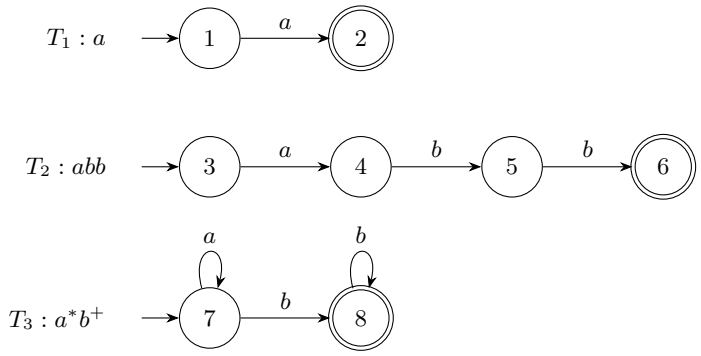
5

Clase 5 - Autómatas y análisis léxico

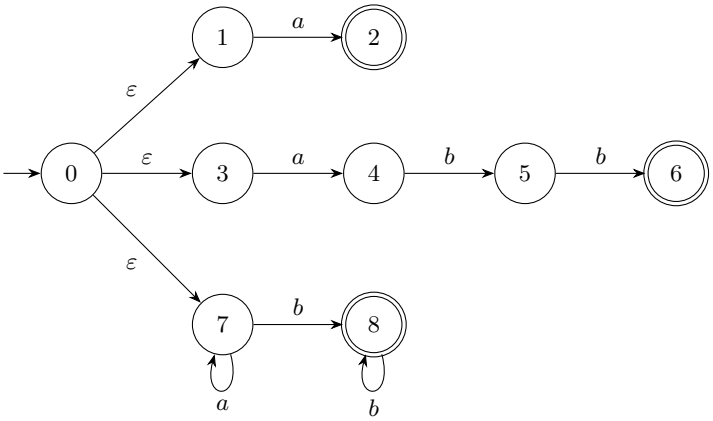
5.1 Expresiones regulares y AFN

5.1.1 Autómatas individuales



5.1.2 Combinación mediante transiciones vacías

Combinamos las tres expresiones mediante transiciones ε.



5.2.1 Aplicación del algoritmo de subconjuntos

$$c_\varepsilon(\{0\}) = \{0,1,3,7\} = A.$$

P/A: $U_a = c_\varepsilon(\text{mover}(A,a)) = c_\varepsilon(\{2,4,7\}) = \{2,4,7\} = B,$
$U_b = c_\varepsilon(\text{mover}(A,b)) = c_\varepsilon(\{8\}) = \{8\} = C.$

P/B: $U_a = c_\varepsilon(\text{mover}(B,a)) = c_\varepsilon(\{7\}) = \{7\} = D,$
$U_b = c_\varepsilon(\text{mover}(B,b)) = c_\varepsilon(\{5,8\}) = \{5,8\} = E.$

P/C: $U_a = c_\varepsilon(\text{mover}(C,a)) = c_\varepsilon(\varnothing) = \varnothing = \text{Error},$
$U_b = c_\varepsilon(\text{mover}(C,b)) = c_\varepsilon(\{8\}) = \{8\} = C.$

P/D: $U_a = c_\varepsilon(\text{mover}(D,a)) = c_\varepsilon(\{7\}) = \{7\} = D,$
$U_b = c_\varepsilon(\text{mover}(D,b)) = c_\varepsilon(\{8\}) = \{8\} = C.$

P/E: $U_a = c_\varepsilon(\text{mover}(E,a)) = c_\varepsilon(\varnothing) = \varnothing = \text{Error},$
$U_b = c_\varepsilon(\text{mover}(E,b)) = c_\varepsilon(\{6,8\}) = \{6,8\} = F.$

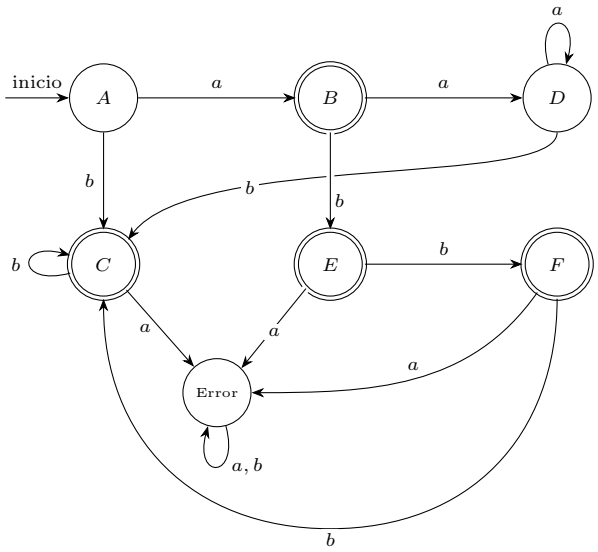
P/F: $U_a = c_\varepsilon(\text{mover}(F,a)) = c_\varepsilon(\varnothing) = \varnothing = \text{Error},$
$U_b = c_\varepsilon(\text{mover}(F,b)) = c_\varepsilon(\{8\}) = \{8\} = C.$

$$D_{\text{estados}} = \{A, B, C, D, E, F, \text{Error}\}.$$

Cuadro 5.1: Tabla de transiciones del AFD

Estado	a	b
A	B	C
B	D	E
C	Error	C
D	D	C
E	Error	F
F	Error	C
Error	Error	Error

Estado inicial: $S_0 = A$. Estados finales: $F = \{B, C, E, F\}$.



5.3 Ambigüedad léxica

El lexema `abb`, reconocido por T_2, también pertenece a T_3, ya que está incluido en a^*b^+. Por lo tanto, en el AFD puede ser reconocido como T_2 o T_3.

Esto se observa en el estado final F: el subconjunto $\{6, 8\}$ contiene los estados finales de T_2 y T_3, respectivamente. Cuando un estado reúne finales de diferentes tokens, existe ambigüedad léxica: no se puede determinar a qué token pertenece el lexema.

Pregunta 5.3.1 ¿En qué caso el AFD de un analizador léxico es ambiguo?

Cuando los patrones no son disjuntos y un mismo lexema puede ser reconocido por más de un token.

5.3.1 Algoritmo para recorrer un AFN con epsilon

Nota 5.3.1 Algoritmo del Libro:

```
S = cerradura_epsilon({S_0});
c = siguienteInput();
while (c != EOF) {
    S = cerradura_epsilon(mover(S, c));
    c = siguienteInput();
}
if (interseccion(S, F) != conjunto_vacio) {
    return true;
} else {
    return false;
}
```

Usando la tabla de transiciones del AFN también podríamos obtener código para el analizador léxico. Sin embargo, como S es un conjunto de estados, su implementación no es tan sencilla: se retornarían varios estados. Con la entrada `abb` terminaríamos en $S = \{6, 8\}$, lo que indica que el lexema pertenece a dos tokens diferentes.

Este algoritmo equivale a una simulación del AFD obtenido mediante el algoritmo de subconjuntos, utilizando cerraduras-ε. Se aplica únicamente a la cadena de entrada, sin construir el AFD completo.

Para la entrada `abb`:

$$S = \{0, 1, 3, 7\},$$
$$S = \{2, 4, 7\},$$
$$S = \{5, 8\},$$
$$S = \{6, 8\}.$$

5.3.2 Detección de ambigüedad léxica

Para encontrar ambigüedad léxica en un conjunto de lenguajes regulares:

1. Construir el AFN de cada lenguaje regular.
2. Combinar los AFN en uno solo mediante transiciones ε.
3. Construir el AFD del AFN combinado.
4. Revisar si algún estado final del AFD contiene estados finales de diferentes tokens. Si es así, existe ambigüedad léxica.

Nota 5.3.2 Observación

Si tenemos una cadena en mente, podemos usar directamente el algoritmo del AFN con ε para comprobar si termina en estados finales de más de un token, sin construir el AFD completo.

5.3.3 ¿Cómo eliminamos la ambigüedad léxica?

Podemos modificar los patrones para que sean disjuntos o establecer una **prioridad entre los tokens** que reconocen el mismo lexema.

En el ejemplo, `abb` pertenece tanto a $T_2 : abb$ como a $T_3 : a^*b^+$. Si damos prioridad a T_2, al llegar al estado $F = \{6, 8\}$ el analizador devuelve T_2. Los demás lexemas de a^*b^+ se siguen reconociendo como T_3.

La elección del **lexema más largo** permite decidir cuánto texto consumir. Cuando dos patrones reconocen el mismo lexema, como ocurre con `abb`, esa regla no basta: se aplica la prioridad definida. **Minimizar el AFD no elimina por sí solo la ambigüedad léxica**.

5.4 Minimización de estados de un AFD

5.4.1 Algoritmo

1. Definir una partición formada por dos grupos: $\Pi = \{S, F\}$, donde S es el conjunto de estados no finales y F el de estados finales.
2. Mientras se obtenga una partición nueva, comparar los estados s, t de cada grupo. Se separan si, para algún símbolo de entrada, llegan a estados que pertenecen a grupos diferentes.
3. El estado inicial es el grupo que contiene al estado inicial del AFD. Los estados finales son los grupos de estados finales.
4. Eliminar los posibles estados inalcanzables.

5.4.2 AFD de la pizarra

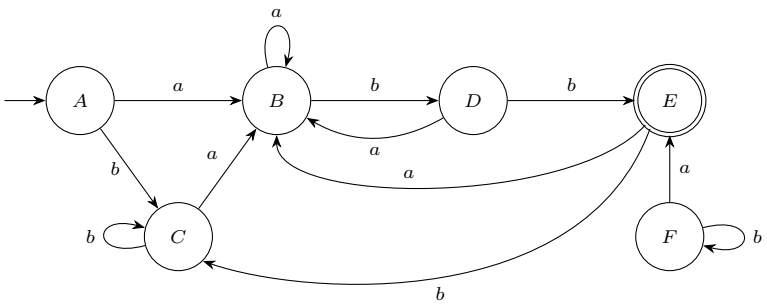


Figura 5.1: AFD de la pizarra

El estado F es inalcanzable desde A y se elimina directamente por la regla 4 del algoritmo. Sus transiciones son $F \xrightarrow{a} E$ y $F \xrightarrow{b} F$. La tabla siguiente y el refinamiento consideran los estados restantes.

Cuadro 5.2: Tabla de transiciones del AFD de la pizarra

Estado	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

$$\Pi = \{F, S\} = \big\{\{E\}, \{A, B, C, D\}\big\}.$$

Grupos: I= $\{E\}$, II= $\{A, B, C, D\}$.

Cuadro 5.3: Grupos con la entrada a

Estado	a	b
A	B_{II}	C
B	B_{II}	D
C	B_{II}	C
D	B_{II}	E
E	B_{II}	C

Cuadro 5.4: Grupos con la entrada b

Estado	a	b
A	B_{II}	C_{II}
B	B_{II}	D_{II}
C	B_{II}	C_{II}
D	B_{II}	E_{I}
E	B_{II}	C_{II}

Se separa D de A, B, C porque con b llega al grupo I.

$$\Pi_{\text{nueva}} = \big\{\{E\}, \{A, B, C\}, \{D\}\big\}.$$

Grupos: I= $\{E\}$, II= $\{A, B, C\}$, III= $\{D\}$.

Cuadro 5.5: Refinamiento con tres grupos

Estado	a	b
A	B_{II}	C_{II}
B	B_{II}	D_{III}
C	B_{II}	C_{II}
D	B_{II}	E_{I}
E	B_{II}	C_{II}

Se separa B de A, C porque con b llega a D, que pertenece al grupo III.

$$\Pi_{\text{nueva}} = \big\{\{E\}, \{A, C\}, \{D\}, \{B\}\big\}.$$

Grupos: I= $\{E\}$, II= $\{A, C\}$, III= $\{D\}$, IV= $\{B\}$.

Cuadro 5.6: Refinamiento con cuatro grupos

Estado	a	b
A	B_{IV}	C_{II}
B	B_{IV}	D_{III}
C	B_{IV}	C_{II}
D	B_{IV}	E_{I}
E	B_{IV}	C_{II}

$$\Pi_{\text{nueva}} = \big\{\{E\}, \{A, C\}, \{D\}, \{B\}\big\}.$$

La partición no cambió; por lo tanto, terminamos. Los estados serán $\{E\}$, $\{A, C\}$, $\{D\}$ y $\{B\}$.

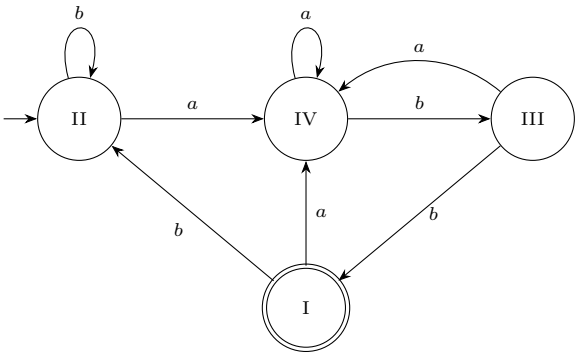
Cuadro 5.7: AFD minimizado: conjuntos de estados

Estado	a	b
$\{A, C\}$	$\{B\}$	$\{A, C\}$
$\{B\}$	$\{B\}$	$\{D\}$
$\{D\}$	$\{B\}$	$\{E\}$
$\{E\}$	$\{B\}$	$\{A, C\}$

O bien, con los rótulos I= $\{E\}$, II= $\{A, C\}$, III= $\{D\}$ y IV= $\{B\}$:

Cuadro 5.8: AFD minimizado: rótulos de grupos

Estado	a	b
II	IV	II
IV	IV	III
III	IV	I
I	IV	II



Nota 5.4.1 Equivalencia de expresiones regulares

Al minimizar el AFD de expresiones regulares, podemos determinar si dos patrones de expresiones regulares son equivalentes, es decir, si reconocen el mismo lenguaje. Si, sobre el mismo alfabeto, los AFD completos y mínimos obtenidos son isomorfos, entonces los patrones son equivalentes: existe una correspondencia entre sus estados que conserva el estado inicial, los estados finales y las transiciones. Los nombres de los estados pueden ser distintos.
Esto puede servir para identificar en expresiones regulares complicadas si definen **patrones IGUALES**.
Con la construcción de Thompson y el algoritmo de subconjuntos podemos obtener autómatas distintos para expresiones regulares equivalentes, aunque esto no ocurre necesariamente en todos los casos. Si los patrones son equivalentes, sus AFD mínimos serán iguales salvo el renombrado de los estados.
La minimización de AFD así funciona como **paso de verificación de nuestros patrones** si tenemos duda.

5.4.3 Extra: minimización del AFD con ambigüedad léxica

Aplicamos el algoritmo al AFD visto anteriormente:

$$\text{II} = \big\{\{B, C, E, F\}, \{A, D, \text{Error}\}\big\}.$$

Grupos: I= $\{B, C, E, F\}$, II= $\{A, D, \text{Error}\}$.

Cuadro 5.9: Partición inicial del AFD combinado

Estado	a	b
A	B_{I}	C_{I}
B	D_{II}	E_{I}
C	$\mathrm{Error}_{\mathrm{II}}$	C_{I}
D	D_{II}	C_{I}
E	$\mathrm{Error}_{\mathrm{II}}$	F_{I}
F	$\mathrm{Error}_{\mathrm{II}}$	C_{I}
Error	$\mathrm{Error}_{\mathrm{II}}$	$\mathrm{Error}_{\mathrm{II}}$

Por la entrada a, se separa A de D, Error: A llega al grupo I y los otros al II. Por la entrada b, se separa D de Error: D llega al grupo I y Error al II.

$$\Pi_{\text{nueva}} = \big\{\{B, C, E, F\}, \{A\}, \{D\}, \{\mathrm{Error}\}\big\}.$$

Grupos: I= $\{B, C, E, F\}$, II= $\{A\}$, III= $\{D\}$, IV= $\{\mathrm{Error}\}$.

Cuadro 5.10: Refinamiento: se consideran B, C, E y F

Estado	a	b
~~A~~	~~B_{I}~~	~~C_{I}~~
B	D_{III}	E_{I}
C	$\mathrm{Error}_{\mathrm{IV}}$	C_{I}
~~D~~	~~D_{III}~~	~~C_{I}~~
E	$\mathrm{Error}_{\mathrm{IV}}$	F_{I}
F	$\mathrm{Error}_{\mathrm{IV}}$	C_{I}
~~Error~~	~~$\mathrm{Error}_{\mathrm{IV}}$~~	~~$\mathrm{Error}_{\mathrm{IV}}$~~

Por la entrada a, se separa B de C, E, F: B llega al grupo III y los otros al IV. Por la entrada b no se separa ningún grupo restante.

$$\Pi_{\text{nueva}} = \big\{\{B\}, \{C, E, F\}, \{A\}, \{D\}, \{\mathrm{Error}\}\big\}.$$

Grupos: I= $\{B\}$, II= $\{C, E, F\}$, III= $\{A\}$, IV= $\{D\}$, V= $\{\mathrm{Error}\}$.

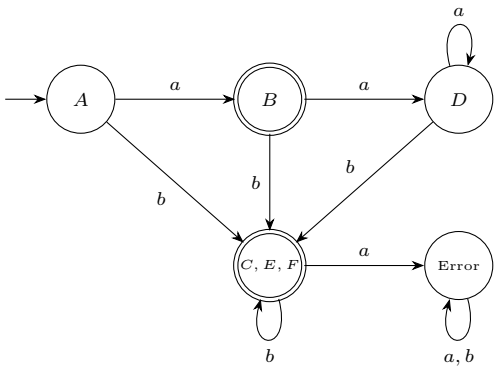
Cuadro 5.11: Refinamiento: solo se consideran C, E y F

Estado	a	b
~~A~~	~~B_{I}~~	~~C_{II}~~
~~B~~	~~D_{IV}~~	~~E_{II}~~
C	$\mathrm{Error}_{\mathrm{V}}$	C_{II}
~~D~~	~~D_{IV}~~	~~C_{II}~~
E	$\mathrm{Error}_{\mathrm{V}}$	F_{II}
F	$\mathrm{Error}_{\mathrm{V}}$	C_{II}
~~Error~~	~~$\mathrm{Error}_{\mathrm{V}}$~~	~~$\mathrm{Error}_{\mathrm{V}}$~~

Ya no se separa ningún grupo por a o b; por lo tanto, terminamos.

Nota 5.4.2 Observación

Tachamos las filas que ya no se van a considerar porque sus estados quedaron solos, para simplificar el chequeo mental.



Los estados finales son $\{B\}$ y $\{C, E, F\}$.

5.5 Conversión entre gramáticas libres de contexto y expresiones regulares

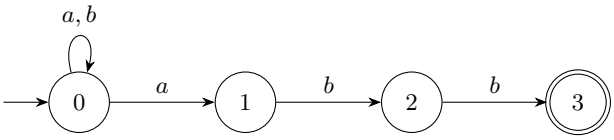
Nota 5.5.1 Definición del libro

Para cada estado de un AFN:

1. Cada estado i se representa mediante una variable sintáctica A_i.
2. Si existe una transición de i a j con la entrada a, se agrega $A_i \to aA_j$. Si la transición es con ε, se agrega $A_i \to A_j$.
3. Si i es final, se agrega $A_i \to \varepsilon$.
4. Si i es inicial, A_i es el símbolo inicial de la gramática en BNF.

5.5.1 Ejemplo del libro

Se considera la expresión regular $(a \mid b)^*abb$:



Se convierte en:

$$A_0 \to aA_0 \mid aA_1 \mid bA_0,$$
$$A_1 \to bA_2,$$
$$A_2 \to bA_3,$$
$$A_3 \to \varepsilon.$$

Este BNF es equivalente a la expresión regular $(a \mid b)^*abb$: se convirtió una definición mediante una expresión regular en una gramática en BNF.

Esto es similar a la estructura de un TDS, que incluye el análisis léxico y semántico en el sintáctico. Podemos tratar los lexemas dentro del analizador sintáctico como producciones de una gramática libre de contexto.

El caso $(0) \xrightarrow{\varepsilon} (0)$ genera $A_0 \to A_0$, que tiene recursión por la izquierda.

El factor común directo o indirecto de la gramática corresponde al no determinismo de la expresión regular. **Al realizar esta conversión de AFN a BNF, debemos verificar que la gramática sea predictiva.**

5.5.2 Aplicación de la conversión de un AFD

Aplicamos la construcción al AFD de la pizarra de la figura 5.1, después de eliminar el estado inalcanzable F:

$$
\begin{aligned}
A_A &\to aA_B \mid bA_C, \\
A_B &\to aA_B \mid bA_D, \\
A_C &\to aA_B \mid bA_C, \\
A_D &\to aA_B \mid bA_E, \\
A_E &\to aA_B \mid bA_C \mid \varepsilon.
\end{aligned}
$$

Un AFD no tendrá recursión por la izquierda en esta construcción, ya que no presenta el caso especial con ε. Tampoco tendrá factor común directo, porque no tiene no determinismo. No tendrá factor común indirecto, ya que todas las producciones no vacías comienzan con un terminal.

Por lo tanto, **la conversión de un AFD a BNF es predictiva**. Podemos convertir un AFN a un AFD y luego a BNF para obtener mecánicamente una gramática predictiva que reconozca el mismo lenguaje, sin preocuparnos por el factor común indirecto.

De esta manera, el análisis léxico y el sintáctico pueden integrarse en un solo analizador predictivo, como en un TDS.

Un problema es cómo representar el lenguaje de entrada mediante una expresión regular, porque puede ser muy grande y complicado. Los lenguajes que pueden describirse mediante expresiones regulares son un subconjunto de los que pueden describirse mediante gramáticas libres de contexto: las expresiones regulares son más restringidas.